

Topic 8 自编码器（Autoencoder）

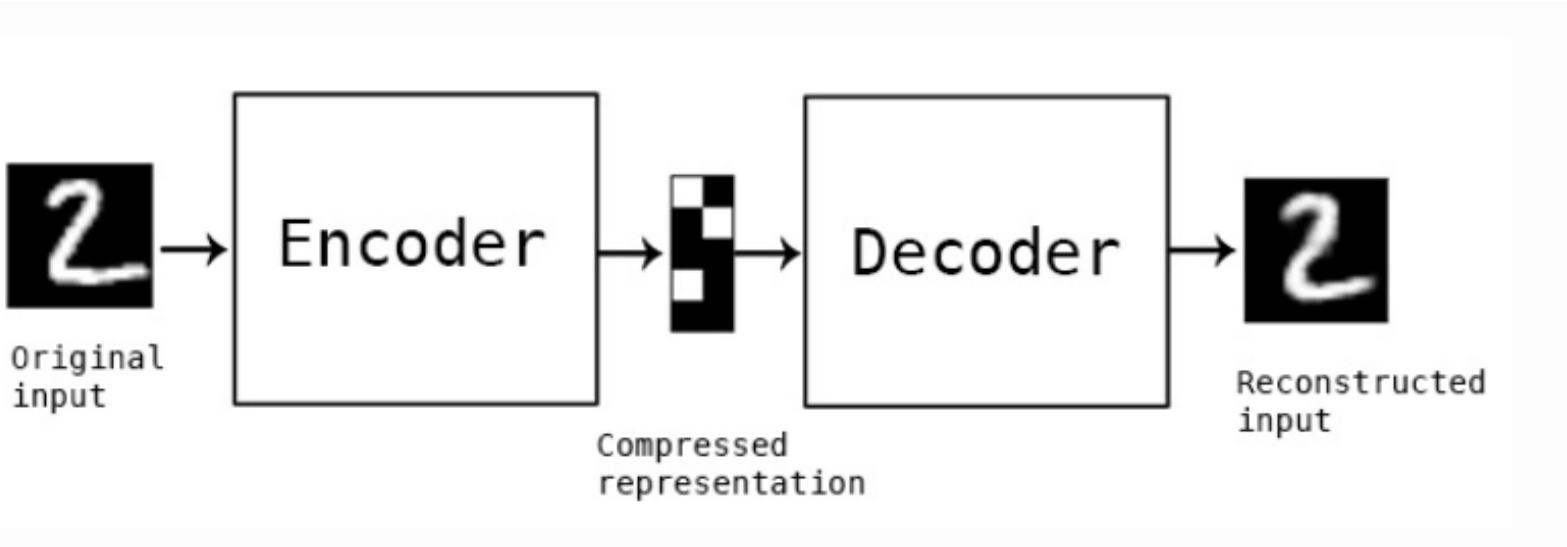
魏准
浙江大学，信息与电子工程学院
2026

总体内容

主要知识点

- 自编码器（Autoencoder）简介
- Keras简介
- 实例：利用Keras训练、测试一个自编码器
- 代码框架简介

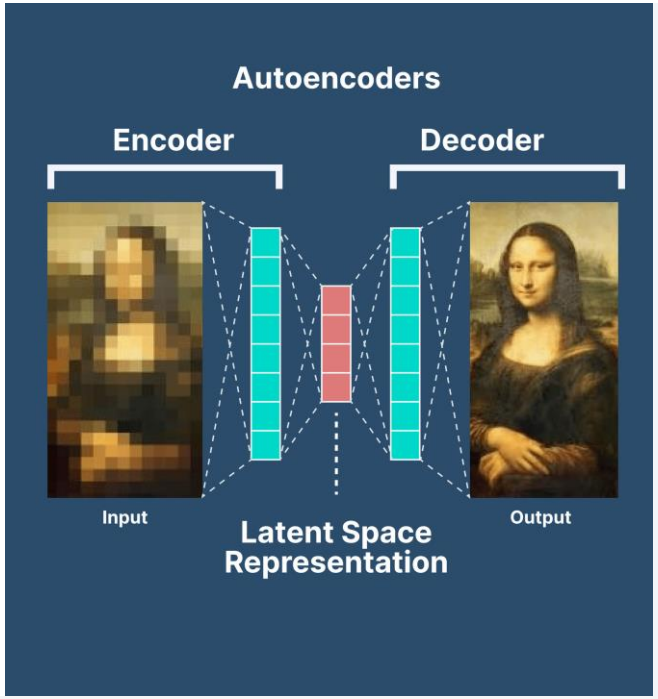
1.1：自编码器（Autoencoder）简介

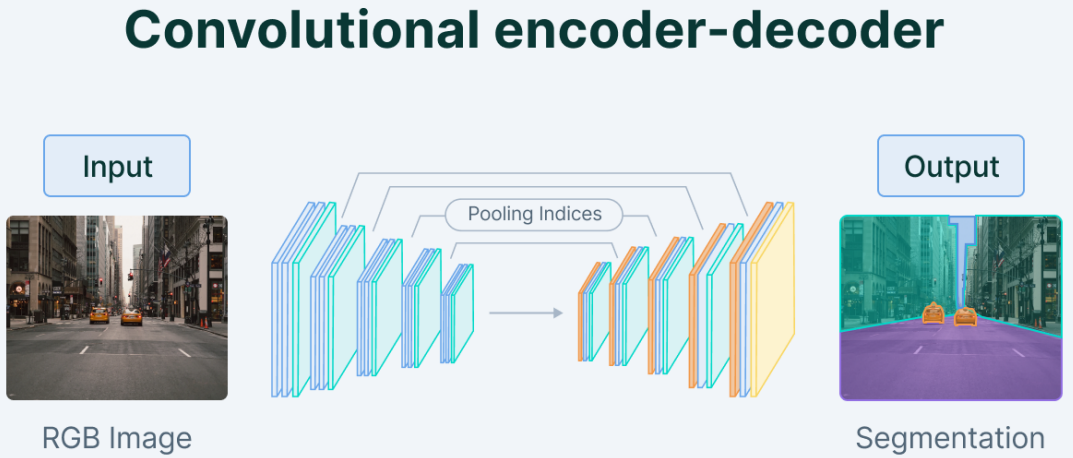


自动编码器是一种数据的压缩算法，在大部分提到自动编码器的场合，压缩和解压缩的函数是通过神经网络实现的。

数据的压缩和解压缩函数是

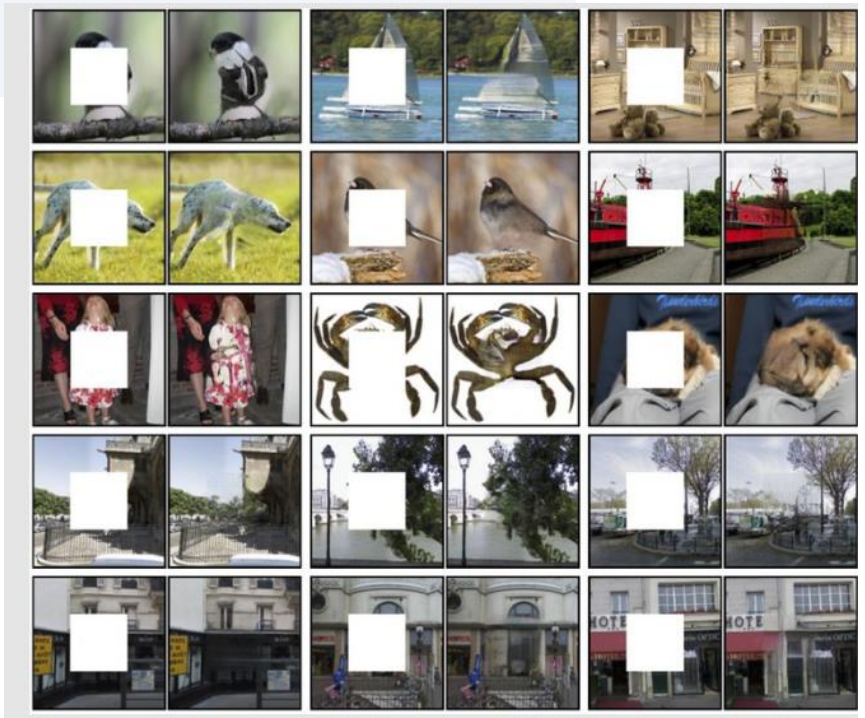
- 数据相关的，意味着自动编码器只能压缩那些与训练数据类似的数据。
- 有损的，意思是解压缩的输出与原来的输入相比是退化的。





自编码器擅长做什么？

- 数据去噪
- 数据降维
- 图像分割
- 图像重建
- 图像修复
- …



Google Scholar

autoencoder

Articles

About 21,500 results (**0.06** sec)

Any time
Since 2023
Since 2022
Since 2019
Custom range...

Sort by relevance
Sort by date

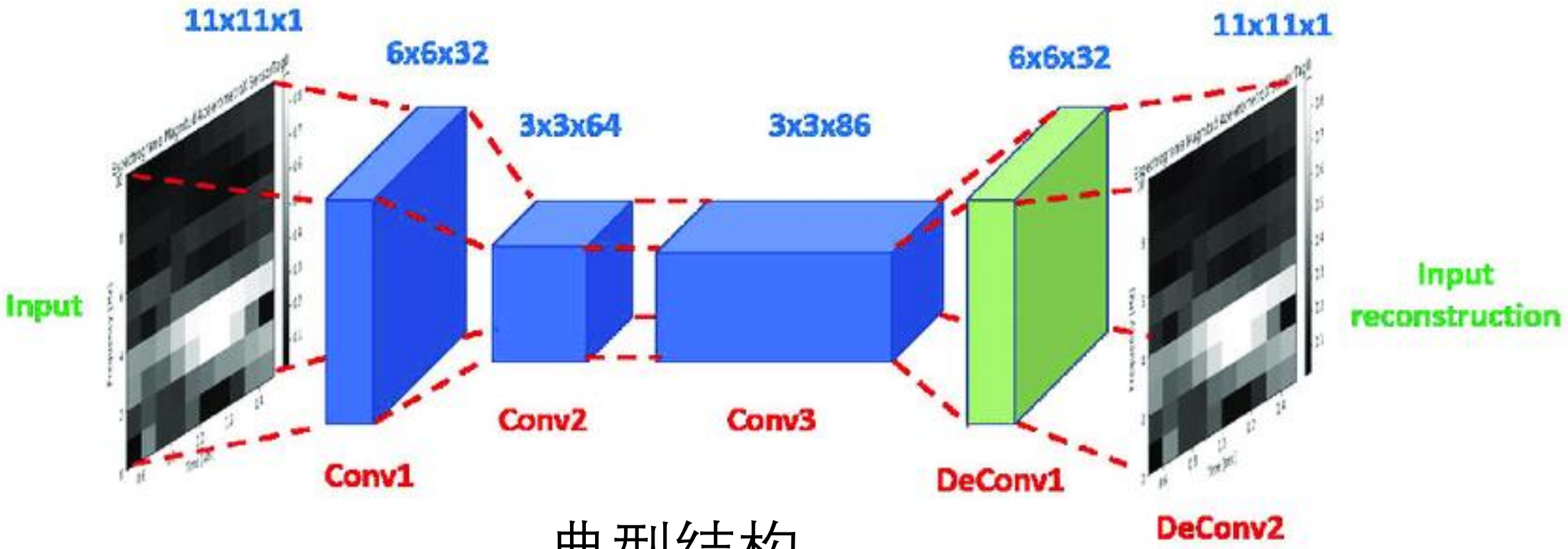
Any type
Review articles

include patents
include citations

A survey of autoencoder-based recommender systems
G Zhang, Y Liu, X Jin - Frontiers of Computer Science, 2020 - Springer
… Applying **autoencoder** on recommender systems would improve the quality of … researches on **autoencoder**-based recommender systems. The differences between **autoencoder**-based …
☆ Save 99 Cite Cited by 86 Related articles All 4 versions

Autoencoder-based representation learning and its application in intelligent diagnosis: A review
Z Yang, B Xu, W Luo, F Chen - Measurement, 2022 - Elsevier
… , especially the representation learning based on **Autoencoder** (AE) theory has been widely … type AEs and the training method of stacked **autoencoder** (SAE) are briefly introduced. Then …
☆ Save 99 Cite Cited by 29 Related articles

Autoencoders
D Bank, N Koenigstein, R Giryes - arXiv preprint arXiv:2003.05991, 2020 - arxiv.org
… This chapter surveys the different types of **autoencoders** that are mainly used today. It also



典型结构

压缩比：

Ratio=(11*11*1)/(3*3*64)

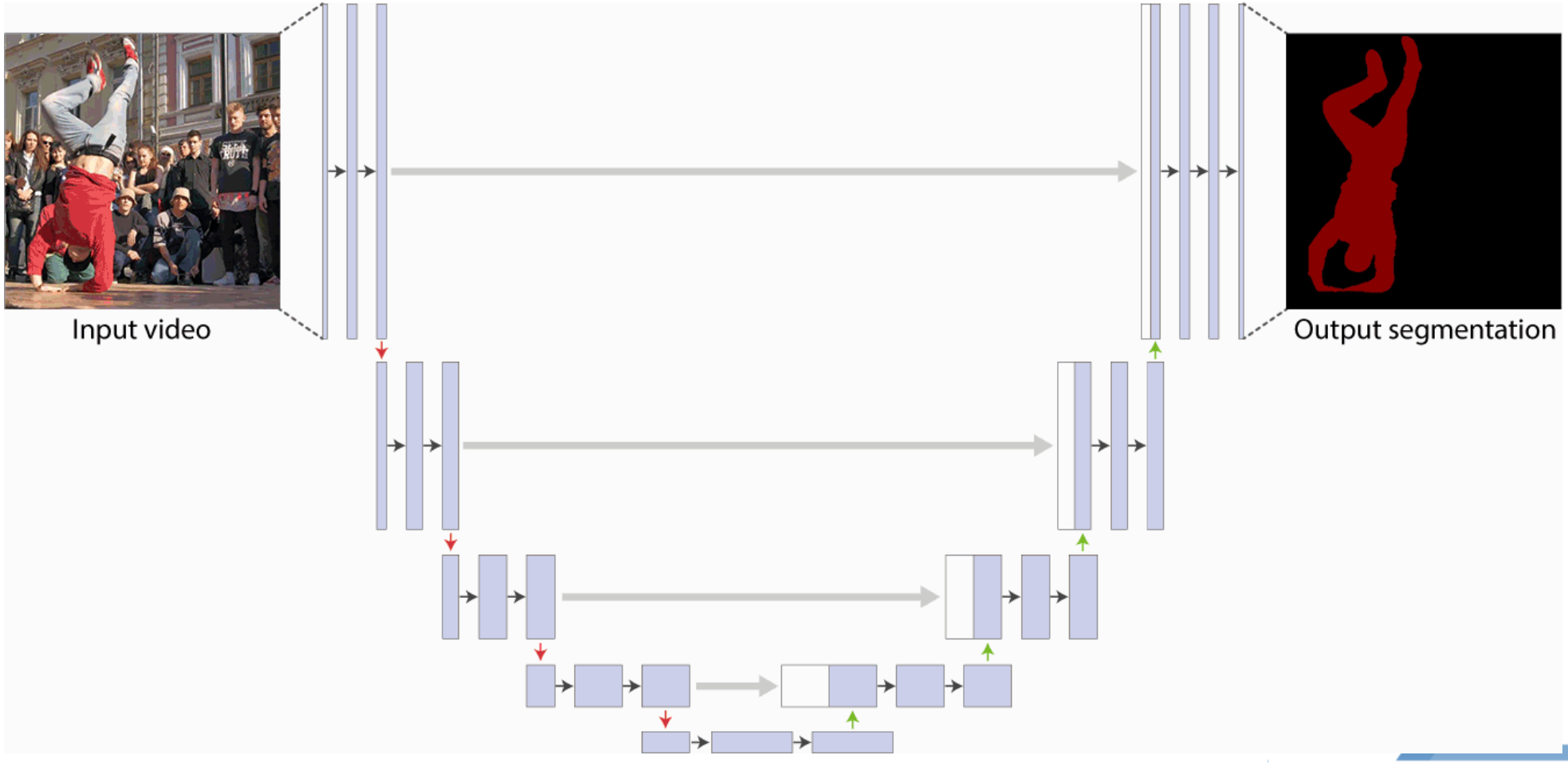
U-NET-Background

U-net: Convolutional networks for biomedical image segmentation

O Ronneberger, P Fischer, T Brox - … , Munich, Germany, October 5-9, 2015 …, 2015 - Springer

There is large consent that successful training of deep networks requires many thousand annotated training samples. In this paper, we present a network and training strategy that relies on the strong use of data augmentation to use the available annotated samples more efficiently. The architecture consists of a contracting path to capture context and a symmetric expanding path that enables precise localization. We show that such a network can be trained end-to-end from very few images and outperforms the prior best method (a sliding …

☆ Save 99 Cite Cited by 62502 Related articles All 29 versions



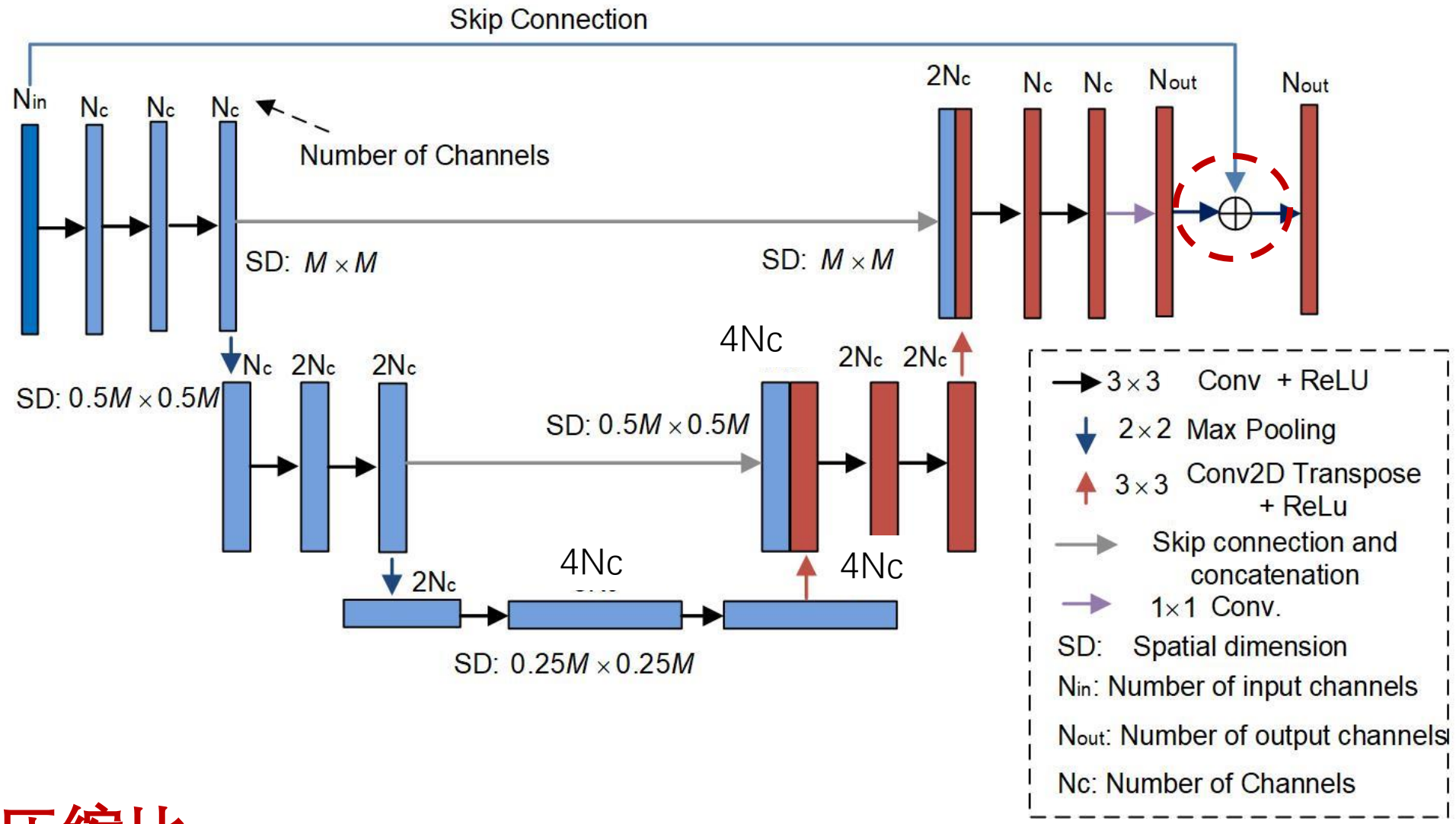
From: Image and Network Analysis Group

U-NET-Background



Image From http://casser.io/files/transfer_learning.html

U-NET



压缩比：

Ratio=(M*M*Nc)/(0.25M*0.25M*4NC)

1.2：Keras简介

Keras是一个**高层**神经网络API（应用程序接口），Keras由纯Python编写而成，且能够以TensorFlow，CNTK，或者 Theano 作为后端运行。Keras 为支持快速实验而生，能够把想法迅速转换为结果：

- 简易和快速的原型设计（keras具有高度模块化，极简，和可扩充特性）
- 无缝CPU和GPU切换
- 用户友好
- 模块性
- 易扩展性

层常用函数：卷积

```
tf.keras.layers.Conv2D(
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    data_format=None,
    dilation_rate=(1, 1),
    groups=1,
    activation=None,
    use_bias=True,
    kernel_initializer="glorot_uniform",
    bias_initializer="zeros",
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

- **filters**: Integer, the dimensionality of the output space (i.e. the number of output filters in the convolution). (K)
- **kernel_size**: An integer or tuple/list of 2 integers, specifying the height and width of the 2D convolution window. Can be a single integer to specify the same value for all spatial dimensions. （$F \times F$）
- **padding**: one of "valid" or "same" (case-insensitive). "valid" means no padding. "same" results in padding with zeros evenly to the left/right or up/down of the input. When padding="same" and strides=1, the output has the same size as the input.
- **activation**: Activation function to use. If you don't specify anything, no activation is applied (see keras.activations).

池化

```
tf.keras.layers.MaxPooling2D(
    pool_size=(2, 2), strides=None, padding="valid", data_format=None, **kwargs
)
```

- **pool_size**: integer or tuple of 2 integers, window size over which to take the maximum. (2, 2) will take the max value over a 2x2 pooling window. If only one integer is specified, the same window length will be used for both dimensions.
- **strides**: Integer, tuple of 2 integers, or None. Strides values. Specifies how far the pooling window moves for each pooling step. If None, it will default to pool_size.
- **padding**: One of "valid" or "same" (case-insensitive). "valid" means no padding. "same" results in padding evenly to the left/right or up/down of the input such that output has the same height/width dimension as the input. Valid by default.
- **data_format**: A string, one of channels_last (default) or channels_first. The ordering of the dimensions in the inputs. channels_last corresponds to inputs with shape (batch, height, width, channels) while channels_first corresponds to inputs with shape (batch, channels, height, width). It defaults to the image_data_format value found in your Keras config file at ~/.keras/keras.json. If you never set it, then it will be “channels_last”. （Channel: D1）

反卷积

```
tf.keras.layers.Conv2DTranspose(
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    output_padding=None,
    data_format=None,
    dilation_rate=(1, 1),
    activation=None,
    use_bias=True,
    kernel_initializer="glorot_uniform",
    bias_initializer="zeros",
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

- **filters**: Integer, the dimensionality of the output space (i.e. the number of output filters in the convolution). (K)
- **kernel_size**: An integer or tuple/list of 2 integers, specifying the height and width of the 2D convolution window. Can be a single integer to specify the same value for all spatial dimensions. ($F \times F$)
- **strides**: An integer or tuple/list of 2 integers, specifying the strides of the convolution along the height and width. Can be a single integer to specify the same value for all spatial dimensions. Specifying any stride value != 1 is incompatible with specifying any dilation_rate value != 1.

- Conv2D输出计算:

输入图片矩阵为：WxW
卷积核大小，kernel_size：FxF
步长strides：S
边界扩充padding的值：P （0/1）

输出大小N*N：N=(W−F+2P)/S+1

- Conv2DTranspose输出计算:

边界扩充padding为Same，输出N*N ： N=W*S

边界扩充padding为Valid，输出N*N ： N= (W-1)*S+F

常用损失函数

Probabilistic losses

- BinaryCrossentropy class
- CategoricalCrossentropy class
- SparseCategoricalCrossentropy class
- Poisson class
- binary_crossentropy function
- categorical_crossentropy function
- sparse_categorical_crossentropy function
- poisson function
- KLDivergence class
- kl_divergence function

Regression losses

- MeanSquaredError class
- MeanAbsoluteError class
- MeanAbsolutePercentageError class
- MeanSquaredLogarithmicError class
- CosineSimilarity class
- mean_squared_error function
- mean_absolute_error function
- mean_absolute_percentage_error function
- mean_squared_logarithmic_error function
- cosine_similarity function
- Huber class
- huber function
- LogCosh class
- log_cosh function

常用优化器

Available optimizers

- SGD
- RMSprop
- Adam
- AdamW
- Adadelta
- Adagrad
- Adamax
- Adafactor
- Nadam
- Ftrl

实验8-2：

- 使用Keras工具包编写自动编码器进行图像去噪：把训练样本用噪声污染，利用Keras工具包编写一个卷积神经网络自编码器，使解码器解码出干净的照片，参考Keras官网函数，通过调研/探索不同网络架构、不同优化器、不同loss等:

  ✓ 搭建卷积神经网络自编码器，尽量取得低的相对误差；
  ✓ 找出相对误差小于50%时能够获得的最大压缩比：size(Input)/ size(Compressed); 打印出最小size的shape，注意包括Channel。

相对误差：Err [inx]=np.linalg.norm(GT_CNN-O_CNN)/np.linalg.norm(GT_CNN)

Step 1:　导入模型

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import *
from tensorflow.keras.layers import *
from tensorflow.keras.optimizers import *
from tensorflow.keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
import numpy as np
```

Step 2:　数据处理

```
## 数据处理
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))
```

Step 3:　数据处理

```
## 添加噪声
noise_factor = 0.5
x_train_noisy = x_train + noise_factor *
np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor *
np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

# 将数组中的元素限制在0, 1之间
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

print(x_test.shape)
print(x_test_noisy.shape)
```

Step 4:　搭建模型

```
## 搭建模型
input_img = Input(shape=(28, 28, 1))

x = Conv2D(32, 3, padding='same', activation='relu',
kernel_initializer='he_normal')(input_img)

x = MaxPooling2D(pool_size=(2, 2))(x)

x =Conv2DTranspose(32, 2, strides=(2, 2), padding='same',
activation='relu', use_bias=False,
kernel_initializer='he_normal')(x)

XXXXXXXXXXXXXXXXXXXXX
```

Step 5:　定义优化器

```
learning_rate = 1e-4
autoencoder = Model(input_img, x)
autoencoder.compile(optimizer = Adam(lr = learning_rate),
loss='mean_squared_error')
# SGD、Adam、RMSprop、Nadam等
```

Step 6:　训练模型

```
## 训练
filepath="Auto-{epoch:02d}-{val_loss:.2f}.hdf5"
model_checkpoint = ModelCheckpoint(filepath,
monitor='val_loss',verbose=1, save_best_only=True, period=2)

history =autoencoder.fit(x_train_noisy, x_train,
                epochs=30,
                batch_size=10,
                shuffle=True,
                validation_data=(x_test_noisy, x_test),
                callbacks=[model_checkpoint],)
```

Step 7:　可视化训练过程

```
## 画出训练过程
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```

Step 8:　典型结果测试、可视化、计算误差

```
Inx_N=3
for inx in range(Inx_N):
    test_inp=x_test_noisy[inx,]
    test_inp=np.reshape(test_inp, (1,)+test_inp.shape)
    results = autoencoder.predict(test_inp,batch_size=1)

XXXXXXXXXXXXX
```